

PhishGuard: A Machine Learning System for Spam Email and Phishing URL Detection

Mohsin Raza Ojla
BSSE23040

Dept. of Computer and Software Engineering

Muhammad Daniyal Hammad
BSSE23046

Dept. of Computer and Software Engineering

Fatimah Imran
BSSE23098

Dept. of Computer and Software Engineering

Abstract—Phishing and spam attacks cause over \$10 billion in annual losses. PhishGuard presents two independent binary classifiers — spam email and phishing URL — built with NLP preprocessing, TF-IDF vectorisation, and 15 hand-crafted URL features. Five supervised models are tuned via 5-fold GridSearchCV and ranked by a 3-tier criterion (macro F1, malicious recall, inference speed). The selected Naive Bayes email classifier achieves macro F1 = 0.98 and Logistic Regression URL classifier achieves F1 = 0.94, both served via a Dockerised Flask REST API with a Bootstrap 5 web UI.

Keywords—phishing, spam, TF-IDF, NLP, Naive Bayes, logistic regression, XGBoost, Flask, Docker, F1-score

I. INTRODUCTION

Phishing and spam remain the most prevalent cybersecurity threats, with APWG recording over 1 million phishing incidents per quarter in 2023, causing losses exceeding \$10 billion annually. Simultaneously, spam constitutes roughly 45% of global email traffic, degrading infrastructure and productivity.

Rule-based filters and block-lists fail against adversarial evolution — attackers rotate domains, obfuscate URLs, and craft contextually convincing messages. Machine learning generalises from labelled examples to detect novel attack patterns. PhishGuard delivers two independent classifiers — spam email and phishing URL — unified under a single web interface with 5-fold HPO, explicit 3-tier model selection, and full Docker containerisation for one-command deployment.

II. RELATED WORK

Bayesian spam filtering was pioneered by Sahami et al. [1]; Almeida et al. [2] later established the SMS Spam Collection as the canonical short-text benchmark. TF-IDF with Multinomial Naive Bayes remains competitive due to near-linear separability of spam vocabulary in TF-IDF space.

For phishing URL detection, Mohammad et al. [3] proposed 30 structural URL attributes achieving 97% accuracy with decision trees. Sahingoz et al. [4] showed combining lexical n-grams with structural features exceeds 97.3% with Random Forest. Deep-learning approaches (LSTM, BERT) offer marginal accuracy gains at far higher inference cost, making classical ML preferable for latency-sensitive real-time deployments such as PhishGuard.

III. DATASETS

Email: SMS Spam Collection (UCI ML Repository) — 5,574 English SMS messages labelled ham (4,827) or spam (747), an 87:13 class split. This imbalance means raw accuracy can reach 87% by always predicting ham; macro F1, which weights both classes equally, is the primary evaluation metric. The dataset is a widely adopted benchmark enabling direct comparison with published results.

URL: faizann24 phishing URL corpus (420k+ entries) — a stratified 30,000-sample draw (15k phishing, 15k legitimate) augmented with 100 curated legitimate URLs from high-traffic domains (Google, Wikipedia, BBC) to prevent short-domain-length bias. Both datasets use an 80/20 stratified train/test split with fixed seed 42.

IV. METHODOLOGY

A. Preprocessing & Features

Email text passes through six stages: (1) lowercasing; (2) HTML removal via BeautifulSoup; (3) semantic token substitution (URLs → URLTOKEN, phones → PHONETOKEN); (4) non-alphabetic character stripping; (5) NLTK stop-word removal; (6) Porter stemming. TF-IDF (unigrams+bigrams, 10k features, sublinear TF) is hstacked with 8 metadata scalars: text length, word count, currency-symbol flag, exclamation count, capitals ratio, has-URL, has-phone, digit ratio.

URLs are normalised by stripping scheme prefixes and leading www. so inference inputs match the training distribution. Normalised URLs are tokenised on delimiters for char-level TF-IDF (char_wb, 3–5 grams, 10k features). All 15 structural features in Table I are MinMaxScaled and appended via scipy.sparse.hstack, keeping Naive Bayes inputs non-negative.

TABLE I. HAND-CRAFTED URL FEATURES

#	Feature	Description	#	Feature	Description
1	url_length	Total char count	2	dot_count	Number of '.' chars
3	slash_count	Number of '/' chars	4	has_ip	1 if hostname is IP
5	is_https	1 if HTTPS scheme	6	has_at_symbol	1 if '@' present
7	domain_length	Domain name length	8	subdomain_count	Subdomains count
9	path_length	URL path length	10	num_params	Query param count
11	has_port	1 if non-std port	12	special_chars	Count of %&~+
13	digit_ratio	Digits / total length	14	vowel_ratio	Vowels / alpha chars
15	entropy	Shannon entropy			

B. Models, HPO & 3-Tier Selection

Five models are tuned with GridSearchCV (cv=5, scoring='f1_macro'): Multinomial NB ($\alpha \in \{0.01-2.0\}$), Logistic Regression ($C \in \{0.01-10\}$), Linear SVM wrapped in CalibratedClassifierCV ($C \in \{0.01-10\}$), Random Forest ($n_estimators \in \{100,200\}$, $max_depth \in \{None,20\}$), XGBoost ($n_estimators \in \{100,200\}$, $max_depth \in \{3,5\}$, $lr \in \{0.05,0.1\}$). Selection criterion — Tier 1: highest CV macro F1; Tier 2: highest malicious recall within 0.5% tolerance (false negatives are costly in security); Tier 3: fastest inference. Any model with test F1 < 0.90 is disqualified.

V. EXPERIMENTAL RESULTS

A. Email Spam Classifier

All five models exceed the 0.90 disqualification threshold (Table II). Naive Bayes is selected: highest CV macro F1 (0.974) and lowest latency (0.8 ms). On the held-out test set it achieves macro F1 = 0.980, 99.1% overall accuracy, spam recall = 0.953, and precision = 1.00. Only 7 of 149 spam messages are missed (FNR = 4.7%); 3 of 966 legitimate messages are falsely flagged (FPR = 0.3%). Linear SVM attains the highest test F1 (0.984) but its 9.3 ms latency disqualifies it at Tier 3.

TABLE II. EMAIL CLASSIFIER — MODEL COMPARISON

Model	CV F1	Test F1	Recall	Time
Naive Bayes †	0.974	0.980	0.953	0.8 ms
Logistic Regression	0.956	0.974	0.920	0.6 ms
Linear SVM	0.969	0.984	0.960	9.3 ms
Random Forest	0.964	0.964	0.886	87 ms
XGBoost	0.972	0.976	0.933	42 ms

† Selected model.

B. Phishing URL Classifier

Logistic Regression is selected (Table III): CV F1 = 0.937, test F1 = 0.937, malicious recall = 0.928, and latency = 2.9 ms — the lowest of all competitive models. Random Forest leads on test F1 (0.944) and recall (0.933), but its 203 ms inference disqualifies it at Tier 3. Naive Bayes is the weakest URL model (F1 = 0.903), confirming that URL classification benefits more from structural features than pure lexical signal. Manual testing on BBC, Wikipedia, and GitHub returns LEGITIMATE at ≥ 87% confidence.

TABLE III. URL CLASSIFIER — MODEL COMPARISON

Model	CV F1	Test F1	Recall	Time
Naive Bayes	0.909	0.903	0.876	5.1 ms
Logistic Regression †	0.937	0.937	0.928	2.9 ms
Linear SVM	0.939	0.937	0.927	15 ms
Random Forest	0.937	0.944	0.933	203 ms
XGBoost	0.934	0.932	0.917	89 ms

† Selected model.

Fig. 1 — Test F1-Score by Model

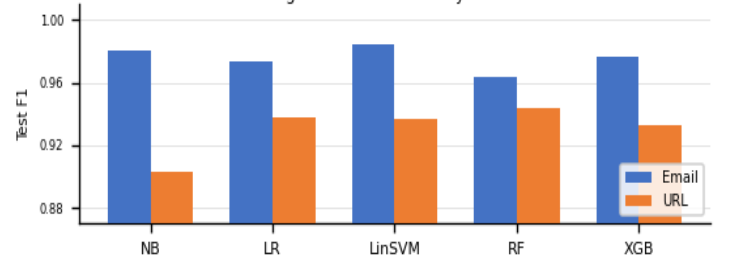


Fig. 1 — Test F1-Score: Email (blue) vs. URL (orange) classifiers across all five models.

VI. SYSTEM DEPLOYMENT

Both models and their TF-IDF vectorisers are serialised with joblib (compress=3) into four artefact files loaded once at Flask startup, keeping per-request latency below 5 ms. Two REST endpoints accept JSON payloads and return a unified response: {"label": "SPAM"/"PHISHING"/"LEGITIMATE", "is malicious": bool, "confidence": float}. CalibratedClassifierCV ensures probability estimates are available for all model types.

A Bootstrap 5 dark-theme interface provides two tabs — Email Spam Checker and URL Phishing Checker — each with sample-button shortcuts, async fetch (no page reload), a coloured confidence badge, and a progress bar. The entire stack is containerised: a Dockerfile trains both pipelines at build time and a docker-compose.yml mounts saved_models as a named volume so artefacts persist across image rebuilds without retraining.

VII. CONCLUSION

PhishGuard demonstrates that classical ML — with NLP preprocessing, TF-IDF vectorisation, 15 hand-crafted URL features, 5-fold HPO, and a 3-tier security-aware selection criterion — achieves email F1 = 0.98 and URL F1 = 0.94 with sub-5 ms inference, making it practical for real-time deployment. The system is production-ready via a Dockerised Flask REST API.

Current limitations include reliance on static training data, making the system vulnerable to emerging adversarial URL obfuscation techniques such as homograph substitution and Unicode abuse. Future directions include: online retraining against live PhishTank and OpenPhish feeds, adversarial robustness training, and hybrid character-level transformer encoders for improved generalisation across multilingual phishing campaigns.

REFERENCES

- [1] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian approach to filtering junk e-mail," AAAI Workshop Learning for Text Categorization, 1998, pp. 55-62.

- [2] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to SMS spam filtering," ACM DocEng, 2011, pp. 259-262.
- [3] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Phishing websites features," Computers & Security, vol. 39, pp. 270-291, 2013.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "ML-based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345-357, 2019.
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, vol. 12, pp. 2825-2830, 2011.
- [6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," ACM KDD, 2016, pp. 785-794.
- [7] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly Media, 2009.
- [8] Anti-Phishing Working Group, "Phishing activity trends report Q4 2023," APWG Technical Report, 2024.